

Casino Game Recommendation & Predictive Modelling

Format Take-home, end-to-end mini-project

Stack Python · dbt (local, dbt-core) · MLflow · scikit-learn

Submission Public GitHub repository (+ optional walkthrough video)

Overview

You work in the data team for an online casino games supplier. The business wants to understand which games are the most suitable for each casino, how casinos group together by behaviour, and how a game is likely to perform at a casino before it goes live there.

You are given three obfuscated tables describing roughly the last 7 days of activity. Using dbt, MLflow and the Python data-science stack, you will model the data, explore it, build game-similarity and casino-cohort views, produce game recommendations for casinos, and train a model that predicts how many players a game will attract at a given casino. You will close with a set of production and business-judgement questions.

We are interested in clear reasoning, choices you can defend, and a tidy, reproducible repository. Where a part leaves the method open, that is deliberate: we want to see the approach you choseD and why. Partial submissions are acceptable; clearly document what is complete and what is not.

Contents

1. Overview
2. Dataset
3. Part 1 — Data Modelling (dbt)
4. Part 2 — Exploratory Data Analysis
5. Part 3 — Game Similarity
6. Part 4 — Casino Cohorts
7. Part 5 — Recommendation System
8. Part 6 — Evaluating the Recommendation System
9. Part 7 — Predicting Player Count
10. Part 8 — Scoring Scenario
11. Part 9 — Production & Operations
12. Part 10 — Business Judgement & Data
13. Deliverables & Repository
14. Submission

Dataset

All data is obfuscated (identifiers and categorical values are anonymised) and covers roughly the last 7 days. The three tables are provided as flat-file extracts (CSV). Build everything downstream from these three raw tables only.

fact_activity — daily player × game × casino activity

One row per date × casino × player × game × currency.

Column	Type	Meaning
DATE	date	Activity date
CASINOID	string	Unique identifier for the casino
PLAYERID	string	Unique identifier for a player
GAMEID	string	Unique identifier for a game
CURRENCYID	string	Unique identifier for a currency
TOTALWAGER	numeric	Total amount wagered that day, in local currency
TOTALPAYOUT	numeric	Total amount paid out that day, in local currency
TOTALSPINS	integer	Number of game rounds played
GOLIVE_DATE	date	Date the game went live at this casino

dim_game — game attributes

Column	Type	Meaning
GAMEID	string	Unique identifier for a game
GAMETYPE	string	Category of game
THEMES	array	Zero or more obfuscated themes
MECHANICS	array	Zero or more obfuscated in-game mechanics

dim_currency — exchange rates

Column	Type	Meaning
CURRENCYID	string	Unique identifier for a currency
EXCHANGE_RATE_TO_BASE	numeric	Multiplier to convert a local amount into the base currency

Definitions

- **Currency conversion:** $\text{amount_base} = \text{amount_local} \times \text{EXCHANGE_RATE_TO_BASE}$. Use the base currency for all monetary metrics.
- **Net win:** $\text{SUM}(\text{TOTALWAGER}) - \text{SUM}(\text{TOTALPAYOUT})$, in base currency.
- **Players on a game at a casino:** $\text{COUNT}(\text{DISTINCT PLAYERID})$ for that game and casino — this is the target for Parts 7 and 8.

Part 1 — Data Modelling (dbt)

Use dbt, running locally with dbt-core (not dbt Cloud), to transform the three raw tables into clean, analysis-ready data models that every later part builds on.

WHAT WE'RE LOOKING FOR

- Build well-structured, sensibly named queries with a clear grain, sourced only from the three raw tables.
- Convert all monetary values to the base currency so figures are comparable across rows and currencies.
- Make the array fields (themes and mechanics) usable for the work that follows.
- Add data validation appropriate to this dataset, and ensure the build is reproducible — re-running it should be safe and consistent.

Deliverable. The dbt project, plus a short README note describing, the grain of the data model, and the dbt syntax on how to run and test it.

Part 2 — Exploratory Data Analysis

Explore the modelled data and report what genuinely matters about it ahead of the modelling work.

WHAT WE'RE LOOKING FOR

- Understand the data — quality issues such as missing values, duplicates, or anomalies — and explain how you treated what you found.
- Examine the distributions and relationships most relevant to the business questions in the later parts.
- Support your analysis with clear, well-labelled visualisations.
- Draw out concrete findings and their implications, rather than only describing the data.

Deliverable. A notebook or script that reproduces your analysis and figures, plus a concise written summary of your findings.

Part 3 — Game Similarity

Determine a way to measure how similar games are to one another, using the game attributes and any other signals you deem relevant.

WHAT WE'RE LOOKING FOR

- Given a game, return the games most similar to it.
- Justify the approach you chose and the trade-offs it involves.
- Sanity-check the output and comment on whether the results look sensible.

Deliverable. The similarity capability (function, table, or other artefact), a few worked examples, and your rationale.

Part 4 — Casino Cohorts

Group casinos into cohorts that share similar preferences and performance.

WHAT WE'RE LOOKING FOR

- Define and justify the features you use to characterise a casino.
- Decide how many cohorts to form and explain how you arrived at that number.
- Describe each cohort in plain language a non-technical reader could follow.

Deliverable. The cohort assignment per casino, the features and method used, and a short profile of each cohort.

Part 5 — Recommendation System

Build a system that recommends the most suitable games to each casino, based on historical performance and preferences.

WHAT WE'RE LOOKING FOR

- Produce a ranked list of recommended games for a given casino, each with a score or a reason.
- Handle games that are already live at a casino in a sensible way.
- Explain the logic behind your approach and its limitations.

Deliverable. The recommender code and ranked output for two or three example casinos, with a short explanation.

Part 6 — Evaluating the Recommendation System

Explain, in writing, how you would evaluate the recommendation system — both during development and once it is running in production.

WHAT WE'RE LOOKING FOR

- Development: how you would measure recommendation quality offline, what you would compare against, and how you would split the data fairly.
- Production: how you would tell whether it is working once live, including the business metrics you would track.
- Risks: the main failure modes or biases you would watch for, and how you would handle the cold-start problem (a new game or a new casino).

Deliverable. A clear written section in your README. Implementing one offline metric is required.

Part 7 — Predicting Player Count

Build a model that predicts the number of players a game will attract at a casino using game similarity together with any other features you engineer.

WHAT WE'RE LOOKING FOR

- Frame the problem clearly, including your unit of observation and how you split the data.
- Take care to avoid leakage when constructing features.
- Compare against a sensible baseline and report, in plain terms, how well your model performs.
- Track every experiment in MLflow — parameters, metrics, and the models — so runs can be compared in the MLflow UI.

Deliverable. Your training code, the MLflow tracking output (add screenshots to your repository), and a short write-up of features, results, and what drives the prediction.

Part 8 — Scoring Scenario

Use your model to predict how many players a specific game would attract at a specific casino.

WHAT WE'RE LOOKING FOR

- Take an example game — its type, themes and mechanics — and a chosen casino, build the model input, and produce a prediction.
- Treat this as the cold-start case in which the game is not yet live at that casino; state your assumptions and how confident you are.

Deliverable. A small runnable function or script and one worked example, with the result explained.

Part 9 — Production & Operations

Answer the following. We are looking for practical reasoning, not a perfect architecture.

- **Deployment & serving:** how would you deploy and serve the recommender and the player-prediction model so that other teams or systems could consume them? Consider batch versus real-time delivery and where each fits.
- **Monitoring & retraining:** how would you monitor these models once they are live, and how would you decide when to retrain them?
- **Operational risk:** what are the main risks of running this system in production, and how would you mitigate them?

Part 10 — Business Judgement & Data

Answer the following in writing.

- **Emerging casinos:** using only this data, how would you identify casinos that are emerging or showing early promise, and what are the pitfalls of those signals?
- **Increasing reach:** how would you use your similarity, cohort and recommendation work to help an emerging casino grow, and what would you measure to know it worked?
- **Additional data:** the dataset provided is deliberately limited. If you could add any further data or fields, what would you add, why, and how would it improve your models or recommendations?

Deliverables & Repository

Your work must be delivered in a public GitHub repository containing, at a minimum: your dbt project; the code and/or notebooks for each part; your MLflow tracking output (or instructions to reproduce it); any figures produced; and a README.

The README should cover:

- How to set up and run your work, from clone to results.
- The key decisions and assumptions you made and why (currency handling, encoding, model and method choices).
- Your EDA findings and headline results.
- How to launch the MLflow UI and what your runs show.
- Your written answers to Parts 6, 9 and 10.
- Known limitations and what you would do with more time.

Submission

1. Push your solution to a public GitHub repository.
2. Ensure your work can be reproduced by following your README.
3. Optionally, include a short (5–10 minute) walkthrough video.
4. Submit the repository URL (and video link, if provided).